

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

ОТЧЁТ
по ознакомительной практике

Выполнил:

Е. К. Федосов

Студент группы
421701

Проверила:

Н. В. Малиновская

Минск 2025

СОДЕРЖАНИЕ

Введение	3
1 Формализация и сравнительный анализ языков представления информации в интеллектуальных системах: LinQ и SPARQL	4
2 Формализация и сравнительный анализ инструментов для разработки интеллектуальных систем: Microsoft Azure AI и Neo4j	12
Заключение	18
Список использованных источников	19

ВВЕДЕНИЕ

Цель:

Провести сравнительный анализ современных технологий разработки интеллектуальных систем, закрепить навыки формализации информации, а также приобрести и развить навыки исследования, анализа и сравнения технологических решений.

Задачи:

1. Изучить языки логического программирования (LinQ и SPARQ) как инструменты формализации декларативных знаний, провести анализ их особенностей, выявить сходства и различия между ними.
2. Рассмотреть современные инструменты для работы с онтологиями и семантическими моделями (Microsoft Azure AI и Neo4j), выявить их функциональные возможности и подходы к представлению знаний, сходства и отличительные особенности.
3. Формализовать полученные результаты в виде SCn-кода.
4. Оформить библиографические источники к полученным результатам по ГОСТ 7.1-2003.

1 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЯЗЫКОВ ПРЕДСТАВЛЕНИЯ ИНФОРМАЦИИ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ: LINQ И SPARQL

LINQ

- :=** [Language Integrated Query]
- :=** [Запрос, интегрированный в язык программирования]
- ∈** язык запросов
- ∈** технология декларативного запроса к данным в .NET
- ⇒** разработчик*:
 - *Microsoft (ведущий архитектор — Эрик Мейер)*
- ⇒** год создания*:
2007
- ⇒** основная парадигма*:
декларативное программирование
- ⇒** характер взаимодействия с данными*:
строго типизированный декларативный синтаксис
- ⇒** описание*:
[LINQ (Language-Integrated Query) — компонент платформы .NET, позволяющий формулировать типобезопасные запросы к различным источникам данных (коллекции, XML, БД) с использованием единого синтаксиса прямо в коде языка программирования (например, C#).]
- ⇒** назначение*:
[LINQ предназначен для упрощения, унификации и безопасного доступа к данным в приложениях .NET. Запросы на LINQ интегрированы в язык и компилируются вместе с основным кодом, обеспечивая единый подход к работе с разнородными источниками данных.]
- ⇒** основные конструкции*:
 - { • [from]
 - :=** [Определяет источник данных для запроса]
 - ⇒** пример*:
[from x in collection select x]
 - ⇒** объяснение*:
[Проходим по каждому элементу x в коллекции и возвращаем его. Это аналог цикла foreach.]
 - [where]
 - :=** [Фильтрация по условию]
 - ⇒** пример*:
[from x in collection where x > 5 select x]
 - ⇒** объяснение*:
[Выбираем только те элементы из коллекции, которые больше 5.]
 - [select]
 - :=** [Определяет результат запроса]
 - ⇒** пример*:
[from x in collection select x.Name]
 - ⇒** объяснение*:
[Из каждого объекта берём только поле Name и включаем его в результат.]

- [group]
 - := [Группировка данных по ключу]
 - ⇒ *пример**:
 - [from x in products group x by x.Category]
 - ⇒ *объяснение**:
 - [Объединяем объекты по значению поля Category. Например, группируем товары по категориям.]
 - [join]
 - := [Объединение двух источников данных по ключу]
 - ⇒ *пример**:
 - [from x in list1 join y in list2 on x.Id equals y.Id
select new {x, y}]
 - ⇒ *объяснение**:
 - [Находим пары элементов из двух списков, где совпадает поле Id.]
 - [orderby]
 - := [Сортировка результата запроса]
 - ⇒ *пример**:
 - [from x in people orderby x.Name ascending select x]
 - ⇒ *объяснение**:
 - [Сортируем коллекцию по имени в алфавитном порядке.]
- ⇒ }
- поддержка**:
- { • [LINQ to Objects]
 - := [Работа с коллекциями и массивами]
 - ⇒ *особенность**:
 - [Позволяет применять запросы к данным в памяти без подключения к БД.]
 - [LINQ to SQL / Entity Framework]
 - := [Интеграция с реляционными базами данных]
 - ⇒ *уточнение**:
 - [Запросы преобразуются в SQL и выполняются на стороне СУБД.]
 - [LINQ to XML]
 - := [Обработка и выборка данных из XML-документов]
 - ⇒ *пример**:
 - [from x in xml.Elements("book") where x.Attribute("id")
== "1" select x]
 - [PLINQ]
 - := [Выполнение LINQ-запросов сразу в нескольких потоках]
 - ⇒ *применение**:
 - [Ускоряет обработку больших коллекций, разбивая её на части и обрабатывая их параллельно.]
 - [IAsyncEnumerable / async LINQ]
 - := [Асинхронная работа с последовательностью данных]
 - ⇒ *применение**:
 - [Позволяет получать и обрабатывать данные по одному, когда они становятся доступны — удобно, если данные приходят постепенно, например, из интернета.]
- }

- ⇒ *особенности**:
- {• [Интеграция в язык программирования]
:= [Запросы являются частью языка и поддерживаются компилятором]
 - [Строгая типизация]
:= [Типы данных проверяются компилятором, и ошибки выявляются на этапе компиляции]
 - [Отложенное выполнение]
:= [Запрос выполняется только при обращении к результату]
 - [Функциональный стиль]
:= [Активное использование лямбда-выражений и цепочек вызовов]
- }
- ⇒ *ограничения**:
- {• [Зависимость от LINQ-провайдеров]
:= [Для различных источников данных требуется реализация LINQ-интерфейса]
 - [Проблемы с трансляцией]
:= [Не все выражения LINQ могут быть преобразованы в SQL]
 - [Ограничения на производительность]
:= [При больших объёмах данных возможна деградация быстродействия без оптимизации]
- }
- ⇒ *реализации**:
- {• [C#]
:= [Основной язык с полной поддержкой LINQ]
 - [F#]
:= [Функциональный стиль запросов с композицией функций]
 - [LINQPad]
:= [Интерактивная среда для написания и тестирования LINQ-запросов]
- }
- ⇒ *примеры использования**:
- {• [Фильтрация данных в коллекциях]
 - [Работа с БД через Entity Framework]
 - [Обработка XML-документов]
 - [Анализ логов и CSV с LINQ to Objects]
 - [Быстрая обработка больших списков через PLINQ]
- }
- ⇒ *библиографические источники**:
- {• [Microsoft Learn. Language Integrated Query (LINQ) [Электронный ресурс] // Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/linq/> (дата обращения: 17.05.2025).]
 - [Wikipedia: LINQ [Электронный ресурс] // Wikipedia. URL: https://en.wikipedia.org/wiki/Language_Integrated_Query (дата обращения: 18.05.2025).]
 - [Metanit: LINQ Tutorial [Электронный ресурс] // Metanit. URL: <https://metanit.com/sharp/tutorial/15.1.php> (дата обращения: 17.05.2025).]
- }

SPARQL

:= [SPARQL Protocol and RDF Query Language]

- := [Протокол и язык запросов к RDF-графам]
- ∈ язык запросов к графовым базам данных
- ⇒ разработчик*:
 - W3C (World Wide Web Consortium)
- ⇒ год создания*:
 - 2008
- ⇒ основная парадигма*:
 - декларативное программирование
- ⇒ характер взаимодействия с данными*:
 - декларативный на основе шаблонов графов
- ⇒ описание*:
 - [SPARQL — официальный язык запросов к данным, представленным в формате RDF (Resource Description Framework). Позволяет формировать гибкие и мощные запросы к графам знаний, извлекать информацию, проверять логические условия и конструировать новые графы.]
- ⇒ назначение*:
 - [Обеспечение возможности извлечения, фильтрации, агрегации и трансформации семантических данных, представленных в виде триплетов RDF. Используется в системах семантического веба, биоинформатике, цифровой гуманитаристике, государственном управлении и др.]
- ⇒ основные конструкции*:
 - {
 - [SELECT]
 - := [Извлечение переменных в табличной форме]
 - ⇒ пример*:
 - [SELECT ?name WHERE { ?person foaf:name ?name }]
 - ⇒ пояснение*:
 - [Этот запрос ищет всех сущностей, у которых указано некоторое имя, и возвращает найденные имена в виде таблицы — по одной строке на каждое совпадение.]
 - [CONSTRUCT]
 - := [Формирование нового RDF-графа на основе шаблона]
 - ⇒ пример*:
 - [CONSTRUCT { ?s foaf:label ?name } WHERE { ?s foaf:name ?name }]
 - ⇒ пояснение*:
 - [Этот запрос создаёт новый RDF-граф, где берёт все объекты, у которых есть имя, и добавляет к ним новое свойство — метку с тем же значением, что и имя.]
 - [ASK]
 - := [Проверка, существует ли хотя бы один результат для шаблона]
 - ⇒ пример*:
 - [ASK WHERE { ?person ex:hasAge ?age . FILTER(?age > 30) }]
 - ⇒ пояснение*:
 - [Возвращает true, если хотя бы один человек в данных имеет возраст более 30 лет.]
 - [DESCRIBE]
 - := [Возврат описания ресурса]
 - ⇒ пример*:

```

    [DESCRIBE <http://example.org/person/123>]
⇒ пояснение*:
    [Возвращает все известные утверждения о заданном ресурсе. И-
      пользуется, когда нужно получить полное описание объекта.]
• [WHERE]
  := [Основной блок условий запроса]
⇒ пример*:
    [SELECT ?name WHERE { ?person ex:hasName ?name }]
⇒ пояснение*:
    [Содержит шаблоны, определяющие, какие данные нужно найти в
      графе.]
• [FILTER]
  := [Ограничение результатов по условиям]
⇒ пример*:
    [FILTER (?age > 30)]
⇒ пояснение*:
    [ Отбрасывает все результаты, не удовлетворяющие условию. Испол-
      зуется совместно с WHERE. ]
• [OPTIONAL]
  := [Добавление необязательных шаблонов к запросу]
⇒ пример*:
    [SELECT ?name ?email WHERE { ?person ex:hasName ?name .
      OPTIONAL { ?person ex:hasEmail ?email } }]
⇒ пояснение*:
    [ Позволяет извлекать данные, если они есть, но не исключает ре-
      зультаты, если их нет. ]
• [UNION]
  := [Объединение результатов нескольких шаблонов]
⇒ пример*:
    [SELECT ?name WHERE { { ?person ex:hasName ?name } UNION
      { ?person ex:hasLabel ?name } }]
⇒ пояснение*:
    [ Объединяет данные из двух разных путей — например, когда ин-
      формация может быть представлена разными свойствами. ]
• [GRAPH]
  := [Запросы к определённомu именованному графу]
⇒ пример*:
    [SELECT ?name WHERE { GRAPH <http://example.org/graph1>
      { ?person ex:hasName ?name } }]
⇒ пояснение*:
    [ Позволяет выполнять запросы к конкретному подграфу в хранили-
      ще с несколькими графами. ]
}
⇒ поддержка*:
{• [Работа с RDF и OWL]
  := [SPARQL работает с триплетами RDF и поддерживает онтологии,
      определённые в OWL]
• [Подсчёты и фильтрация]
  :=

```


[Поддержка функций для подсчёта и вычислений — таких как количество (COUNT), сумма (SUM), среднее (AVG), минимум (MIN) и максимум (MAX)]

- [Федеративные запросы (SERVICE)]
:= [Позволяют выполнять запросы к другим внешним базам данных, которые поддерживают SPARQL]
- [Обновление графов (SPARQL Update)]
:= [Возможность добавления и удаления записей из RDF-базы данных]
⇒ *операции**:
[INSERT, DELETE, DELETE WHERE, INSERT DATA]

}

⇒ *применение**:

- *семантический веб*
- *графы знаний*
- *биоинформатика*
- *цифровые гуманитарные науки*
- *государственные открытые данные*
- *объединение разных баз данных*

⇒ *примеры использования**:

- { • [Извлечение данных из Wikidata]
- [Конструирование новых графов (CONSTRUCT)]
- [Проверка наличия информации (ASK)]
- [Объединение запросов через UNION]
- [Использование FILTER для условий]
- [Обращение к внешним графам через SERVICE]

}

⇒ *реализации**:

- { • [Apache Jena]
:= [Java-фреймворк для работы с RDF, OWL, SPARQL]
- [Virtuoso]
:= [Масштабируемая база данных для хранения RDF-данных (триплетов)]
- [GraphDB]
:= [Коммерческое RDF-хранилище с поддержкой логического вывода (reasoning) на основе онтологий]
- [Blazegraph]
:= [Высокопроизводительное open-source RDF-хранилище, использовавшееся в Wikidata]

}

⇒ *особенности**:

- { • [Работа с графовой моделью данных (RDF)]
- [Совместимость с OWL-онтологиями]
- [Мощная фильтрация и логическая проверка]
- [Поддержка именованных графов]
- [Возможность объединения распределённых данных]

}

⇒ *ограничения**:

- { • [высокий порог входа для новичков]
- [необходимость знания онтологического моделирования]
- [производительность зависит от используемого RDF-хранилища]

- [без дополнительных инструментов сложно делать сложные логические выводы]
- ⇒ } библиографические источники*:
- {• [Wikipedia: SPARQL [Электронный ресурс] // Wikipedia. URL: <https://en.wikipedia.org/wiki/SPARQL> (дата обращения: 19.05.2025).]
 - [W3C: SPARQL 1.1 Query Language [Электронный ресурс] // W3C Recommendation. URL: <https://www.w3.org/TR/sparql11-query/> (дата обращения: 20.05.2025).]
 - [Ontotext: What Is SPARQL? [Электронный ресурс] // Ontotext. URL: <https://www.ontotext.com/knowledgehub/fundamentals/what-is-sparql/> (дата обращения: 19.05.2025).]
 - [Wikibooks: SPARQL Tutorial [Электронный ресурс] // Wikibooks. URL: <https://en.wikibooks.org/wiki/SPARQL> (дата обращения: 21.05.2025).]
- ⇒ }

Сравнение языков запросов *LINQ* и *SPARQL*

- = {• *LINQ*
• *SPARQL*
}

⇒ сходства*:

- {• [Оба языка реализуют декларативный подход к извлечению и обработке данных]
 - [Поддерживают работу с различными источниками данных (графы, коллекции, базы)]
 - [Имеют собственный набор конструкций: фильтрация, проекция, группировка]
 - [Используются в системах, связанных с анализом и интеграцией информации]
 - [Поддерживают отложенное выполнение запросов и оптимизацию исполнения]
- ⇒ }

⇒ различия*:

- {• [LINQ встроен в языки .NET-платформы (например, C# или F#), тогда как SPARQL — самостоятельный язык запросов, предназначенный для RDF-графов]
 - [LINQ ориентирован на работу с объектами, XML-документами и данными из таблиц баз данных, тогда как SPARQL работает с графами данных и структурами, основанными на связях между сущностями и их атрибутами.]
 - [LINQ строго типизирован и интегрирован в язык программирования, тогда как SPARQL более гибкий, но требует знания RDF-схем]
 - [SPARQL применяется преимущественно в задачах семантического веба и графовых базах, LINQ — в рамках типичных .NET-приложений]
 - [LINQ зависит от LINQ-провайдеров, SPARQL — от производительности RDF-хранилищ]
 - [LINQ использует C#-подобный синтаксис, SPARQL — синтаксис, близкий к SQL]
- ⇒ }

⇒ рекомендации по выбору*:

- {• [LINQ рекомендуется разработчикам .NET для обработки объектов, XML-

- документов и данных из SQL-баз в рамках приложений на C# или F#]
 - [LINQ удобен для задач фильтрации, агрегации и анализа коллекций в приложениях]
 - [SPARQL является предпочтительным выбором для работы с графами знаний, онтологиями и хранилищами RDF]
 - [SPARQL незаменим в проектах семантического веба, биоинформатики, цифровой гуманитаристики и информационного поиска с учётом семантики]
- }

2 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ИНСТРУМЕНТОВ ДЛЯ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ: MICROSOFT AZURE AI И NEO4J

Microsoft Azure AI

- :=** [Платформа облачных ИИ-сервисов, предоставляемая Microsoft]
- ∈** *инструмент для разработки интеллектуальных систем*
- ∈** *облачная вычислительная платформа*
- ⇒** *разработчик**:
Microsoft
- ⇒** *год запуска**:
2017
- ⇒** *модель лицензирования**:
оплата по использованию (pay-as-you-go)
- ⇒** *назначение**:
[Обеспечение широкого набора инструментов и API для построения, обучения, развёртывания и мониторинга моделей искусственного интеллекта в облаке.]
- ⇒** *функциональные возможности**:
 - {**• [Azure Machine Learning]
 - :=** [Среда для обучения, мониторинга ML/AI моделей]
 - ⇒** *уточнение**:
[Используется для MLOps, автоматизации и масштабируемого развёртывания моделей.]
 - ⇒** *ключевые компоненты**:
 - {**• [AutoML]
 - :=** [Автоматический выбор моделей и гиперпараметров]
 - [Model Registry]
 - :=** [Хранилище и контроль версий моделей]
 - [Pipeline Designer]
 - :=** [Инструмент для визуального проектирования процессов обучения и инференса]
 - }**
 - [Cognitive Services]
 - :=** [Набор API для обработки изображений, речи, текста и принятия решений]
 - ⇒** *пояснение**:
[Позволяет использовать ИИ без необходимости обучения моделей.]
 - ⇒** *подсервисы**:
 - {**• [Computer Vision]
 - [Text Analytics]
 - [Speech-to-Text / Text-to-Speech]
 - [Language Translation]
 - }**
 - [Azure OpenAI Service]
 - :=** [Доступ к крупным языковым моделям OpenAI (GPT-4, Codex, DALL·E)]

- ⇒ *пояснение**:
 - [Позволяет использовать генеративные ИИ-модели через интернет-запросы, с защитой и отслеживанием действий, как в других сервисах Azure.]
 - [Azure AI Studio]
 - := [Интегрированная среда управления проектами ИИ]
 - ⇒ *примечание**:
 - [Объединяет интерфейсы для загрузки данных, подготовки, аннотации, обучения и публикации моделей.]
 - [Azure AI Search]
 - := [Интеллектуальный поиск с семантическим индексированием]
 - ⇒ *особенность**:
 - [Позволяет автоматически находить в тексте ключевые фразы, имена людей, организаций, мест и проводить общий языковой анализ.]
- }
- ⇒ *поддерживаемые технологии**:
- {
 - [Native Graph Storage]
 - [CSV Import]
 - [JSON / XML]
 - [Neo4j Desktop]
 - [Neo4j Aura (облако)]
 - [Docker-контейнеры]
 - [Kubernetes-деплой]
 - [Neo4j Bloom]
 - := [Визуальный интерфейс для исследования графа]
 - [Graph Data Science Library]
 - := [Инструменты для построения признаков, рекомендаций и анализа сетей]
- }
- ⇒ *преимущества**:
- {
 - [масштабируемость и отказоустойчивость]
 - [гибкая настройка процессов обучения моделей и их применения в рабочих системах]
 - [интеграция с другими сервисами Azure]
 - [поддержка всей цепочки ML-процесса (данные → обучение → деплой)]
 - [доступ к современным языковым и визуальным моделям (GPT, CLIP и др.)]
 - [встроенная поддержка CI/CD и DevOps-процессов]
 - [сертификации по стандартам безопасности (ISO, GDPR и др.)]
- }
- ⇒ *недостатки**:
- {
 - [зависимость от облачной инфраструктуры]
 - [необходимость оплаты для полноценного использования]
 - [необходимость освоения облачных и DevOps-подходов]
 - [ограниченная доступность некоторых сервисов в отдельных регионах]
- }
- ⇒ *библиографические источники**:
- {
 - [Microsoft Azure AI documentation [Электронный ресурс] // Microsoft Learn. URL: <https://learn.microsoft.com/en-us/azure/ai-services/> (дата обращения: 19.05.2025).]
 -

[Azure Machine Learning overview [Электронный ресурс] // Microsoft Learn. URL: <https://learn.microsoft.com/en-us/azure/machine-learning/> (дата обращения: 20.05.2025).]

- [Azure OpenAI Service [Электронный ресурс] // Microsoft Learn. URL: <https://learn.microsoft.com/en-us/azure/cognitive-services/openai/> (дата обращения: 20.05.2025).]

}

Neo4j

:= [Графовая база данных с открытым исходным кодом]

:= [СУБД, ориентированная на хранение и анализ графовых структур, включая графы знаний]

∈ *инструмент для разработки интеллектуальных систем*

∈ *графовая СУБД*

⇒ *разработчик**:

Neo4j, Inc.

⇒ *год запуска**:

2007

⇒ *модель лицензирования**:

Community (бесплатная) и Enterprise (коммерческая)

⇒ *назначение**:

[Оптимизированное хранение, визуализация и выполнение запросов к графам знаний и семантическим сетям. Широко используется для построения систем с логическими связями между объектами: от рекомендательных и экспертных систем до инструментов анализа социальных графов.]

⇒ *функциональные возможности**:

{ • [Cypher Query Language]

:= [Декларативный язык для запросов к графам]

⇒ *особенность**:

[Позволяет описывать паттерны в связях между сущностями естественным образом.]

• [Graph Algorithms]

:= [Набор встроенных алгоритмов графового анализа]

⇒ *применение**:

[Используются для извлечения знаний из структуры графа.]

⇒ *алгоритмы**:

{ • [PageRank]

:= [Метод оценки значимости узлов по числу и весу входящих связей]

• [Louvain]

:= [Алгоритм для поиска сообществ в графах (кластеризация)]

• [Поиск кратчайшего пути (Dijkstra)]

:= [Поиск оптимального пути между узлами по взвешенным рёбрам]

• [Связность компонент]

:= [Обнаружение независимых (изолированных) подграфов]

}

• [Graph Visualization]

```

:=      [Интерактивное отображение узлов и связей]
⇒      инструменты*:
        [Neo4j Bloom, встроенный браузер]
⇒      особенность*:
        [Позволяет исследовать структуру графа в реальном времени.]
•      [Graph Data Science (GDS)]
:=      [Библиотека для анализа графов и ML-задач]
⇒      возможности*:
        [Поддержка обучения моделей на графах, извлечение признаков,
        прогноз связей.]
•      [Гибкая интеграция]
:=      [Подключение к внешним приложениям и языкам программирова-
        ния]
⇒      языки и технологии*:
        {•      Python
          •      Java
          •      GraphQL
          •      REST API
        }
      }
⇒      поддерживаемые технологии*:
      {•      [Neo4j Bloom]
        :=      [Визуальный интерфейс для исследования графа]
        •      [Graph Data Science Library]
        :=      [Инструменты для построения признаков, рекомендаций и анализа]
        •      [Native Graph Storage]
        :=      [Формат хранения, специально разработанный для эффективной ра-
        боты с графовыми данными и запросами]
        •      [CSV Import]
        :=      [Загрузка табличных данных в графовую базу через CSV-файлы]
        •      [JSON / XML]
        :=      [Форматы для хранения и передачи структурированных данных]
        •      [Neo4j Desktop]
        :=      [Локальное приложение для работы с экземплярами Neo4j и проек-
        тами]
        •      [Neo4j Aura]
        :=      [Облачный хостинг Neo4j с автоматическим масштабированием и
        резервным копированием]
        •      [Docker]
        :=      [Официальные образы Neo4j для развёртывания в контейнерах]
        •      [Kubernetes]
        :=      [Масштабируемое развёртывание Neo4j в среде Kubernetes]
      }
⇒      преимущества*:
      {•      [высокая скорость обработки запросов к сложным графам]
        •      [поддержка масштабирования и кластеризации (в Enterprise)]
        •      [гибкий язык запросов Cypher]
        •      [интерактивная визуализация]
        •      [широкая поддержка языков и API]
        •      [активное сообщество и документация]
      }

```

- ⇒ }
 - ⇒ *недостатки**:
 - [ограниченная производительность при больших объемах транзакций]
 - [не подходит для хранения и анализа табличных (реляционных) данных]
 - [расширенные возможности доступны только в коммерческой версии]
 - [обладает высоким порогом входа для новичков]
 - ⇒ }
 - ⇒ *библиографические источники**:
 - [Neo4j Documentation [Электронный ресурс] // Официальный сайт. URL: <https://neo4j.com/docs/> (дата обращения: 22.05.2025).]
 - [Cypher Query Language [Электронный ресурс] // Neo4j Developer Portal. URL: <https://neo4j.com/developer/cypher/> (дата обращения: 21.05.2025).]
 - [Neo4j Graph Data Science [Электронный ресурс] // Neo4j. URL: <https://neo4j.com/product/graph-data-science/> (дата обращения: 21.05.2025).]

Сравнение инструментов разработки интеллектуальных систем: Microsoft Azure AI и Neo4j

- = {
 - *Microsoft Azure AI*
 - *Neo4j*
- ⇒ }
 - ⇒ *сходства**:
 - [Оба инструмента предназначены для разработки интеллектуальных систем и реализуют поддержку ИИ-задач]
 - [Поддерживают машинное обучение и анализ данных]
 - [Обладают масштабируемыми архитектурами и поддержкой облачной/локальной интеграции]
 - [Предоставляют API и средства визуализации для работы с данными]
 - [Имеют активное сообщество, официальную документацию и профессиональную поддержку]
 - ⇒ }
 - ⇒ *различия**:
 - [Microsoft Azure AI — облачная платформа с множеством сервисов ИИ, Neo4j — специализированная графовая база данных]
 - [Azure AI ориентирован на работу с ML-моделями и API, тогда как Neo4j — на графовые структуры и язык запросов Cypher]
 - [Azure включает AutoML, OpenAI, Cognitive Services, в то время как Neo4j фокусируется на алгоритмах графового анализа]
 - [Azure использует REST API, Python SDK и другие инструменты, тогда как Neo4j применяет декларативный язык запросов Cypher.]
 - [Azure требует навыков работы с облачными средами и подписки, Neo4j можно использовать бесплатно в редакции Community Edition.]
 - [Neo4j предпочтительнее для задач, связанных с графовой природой данных, тогда как Azure — для создания масштабируемых и комплексных ИИ-решений]
 - ⇒ }
 - ⇒ *рекомендации по выбору**:
 - [Microsoft Azure AI рекомендуется для создания масштабируемых AI/ML решений в облаке с использованием AutoML, OpenAI и Cognitive Services]
 - [Azure AI — идеальный выбор для организаций, ориентированных на быстрое

- развертывание ИИ и интеграцию с другими Azure-сервисами]
 - [Neo4j идеально подходит для проектов, связанных с анализом сложных графов: социальных сетей, графов знаний]
 - [Neo4j эффективен в задачах, где приоритетом является моделирование логических и семантических связей между сущностями]
- }

ЗАКЛЮЧЕНИЕ

В ходе проведённого исследования был осуществлён сравнительный анализ современных технологий разработки интеллектуальных систем: языков запросов к данным и инструментов построения и развертывания интеллектуальных решений.

В первом разделе работы были рассмотрены языки декларативных запросов LINQ и SPARQL. Проанализированы их синтаксические особенности, основные конструкции и сферы применения. Установлено, что LINQ ориентирован на объектно-ориентированную экосистему .NET и предоставляет универсальный подход к работе с различными типами источников данных. В то же время SPARQL предназначен для работы с RDF-графами и онтологиями в рамках Semantic Web и обеспечивает мощные средства для формулирования запросов к распределённым семантическим данным. Результаты анализа были формализованы с помощью SCn-кода.

Во втором разделе были исследованы инструменты Microsoft Azure AI и Neo4j. Платформа Azure AI предоставляет облачную инфраструктуру и широкий набор сервисов для построения, обучения и развёртывания моделей искусственного интеллекта, включая языковые модели, когнитивные API и инструменты автоматизации ML. В свою очередь, Neo4j является графовой базой данных, специально предназначенной для хранения, визуализации и анализа семантических связей и графов знаний. Оба инструмента имеют ярко выраженные преимущества и используются для различных классов задач. Информация о каждом инструменте также была формализована на SCn-коде.

Полученные результаты могут быть использованы при проектировании гибридных интеллектуальных систем, сочетающих возможности облачных вычислений и графового анализа, а также при выборе технологий для решения задач анализа данных, построения онтологий и обработки знаний в интеллектуальных приложениях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Microsoft learn. language integrated query (linq). — 2025. — [Электронный ресурс]. Дата обращения: 17.05.2025. <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>.

[2] Wikipedia: Language integrated query. — 2025. — [Электронный ресурс]. Дата обращения: 18.05.2025. https://en.wikipedia.org/wiki/Language_Integrated_Query.

[3] Metanit: Linq tutorial. — 2025. — [Электронный ресурс]. Дата обращения: 17.05.2025. <https://metanit.com/sharp/tutorial/15.1.php>.

[4] Wikipedia: Sparql. — 2025. — [Электронный ресурс]. Дата обращения: 19.05.2025. <https://en.wikipedia.org/wiki/SPARQL>.

[5] W3c: Sparql 1.1 query language. — 2025. — [Электронный ресурс]. Дата обращения: 20.05.2025. <https://www.w3.org/TR/sparql11-query/>.

[6] Ontotext: What is sparql? — 2025. — [Электронный ресурс]. Дата обращения: 19.05.2025. <https://www.ontotext.com/knowledgehub/fundamentals/what-is-sparql/>.

[7] Wikibooks: Sparql tutorial. — 2025. — [Электронный ресурс]. Дата обращения: 21.05.2025. <https://en.wikibooks.org/wiki/SPARQL>.

[8] Microsoft azure ai documentation. — 2025. — [Электронный ресурс]. Дата обращения: 19.05.2025. <https://learn.microsoft.com/en-us/azure/ai-services/>.

[9] Azure machine learning overview. — 2025. — [Электронный ресурс]. Дата обращения: 20.05.2025. <https://learn.microsoft.com/en-us/azure/machine-learning/>.

[10] Azure openai service. — 2025. — [Электронный ресурс]. Дата обращения: 20.05.2025. <https://learn.microsoft.com/en-us/azure/cognitive-services/openai/>.

[11] Neo4j documentation. — 2025. — [Электронный ресурс]. Дата обращения: 22.05.2025. <https://neo4j.com/docs/>.

[12] Cypher query language. — 2025. — [Электронный ресурс]. Дата обращения: 21.05.2025. <https://neo4j.com/developer/cypher/>.

[13] Neo4j graph data science. — 2025. — [Электронный ресурс]. Дата обращения: 21.05.2025. <https://neo4j.com/product/graph-data-science/>.